CSCI 4210 — Operating Systems
Homework 4 (document version 1.3)
Fully Synchronized Multi-Threaded TCP Server

- **(v1.2)** This homework is due by 10:00PM on Wednesday, April 29, 2026
- You can use at most **two** late days on this assignment
- This homework is to be done individually, so **please do not share your code with others**
- Place all of your code in `hw4.c` for submission; no other files can be submitted
- You **must** use C for this assignment, and all submitted code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; we will also use `-Werror`, which will treat all warnings as critical errors
- You **must** use the POSIX thread (Pthread) library
- All submitted code **must** successfully compile and run on Submitty, which currently uses Ubuntu v22.04.5 LTS and `gcc` version 11.4.0 (`Ubuntu 11.4.0-1ubuntu1~22.04.2`)
- For test cases involving `valgrind`, Submitty uses version 3.18.1
- You will have **eight** penalty-free submissions on Submitty, after which points will slowly be deducted, i.e., `-1` on submission #9, etc.
- You will have at least **three** days before the due date to submit your code to Submitty; if the auto-grading is not available three days before the due date, the due date will be 10:00PM three days after auto-grading becomes available

Hints and reminders

To succeed in this course, do **not** rely on program output to show whether your code is correct. Consistently allocate **exactly** the number of bytes you need regardless of whether you use static or dynamic memory allocation. Further, deallocate dynamically allocated memory via `free()` at the earliest possible point in your code.

Make use of `valgrind` (or `drmemory` or other dynamic memory checkers) to check for errors with dynamic memory allocation and dynamic memory usage. As another helpful hint, close open file descriptors as soon as you are done using them.

Finally, always read (and re-read) the `man` pages for library functions (section 3), system calls (section 2), etc. To better understand how `man` pages are organized, check out the `man` page for `man` itself via `man man`!

Homework specifications

In this fourth and final homework assignment, you will use C to implement a single-process multi-threaded TCP server that implements rudimentary bigram-based predictive phrase generation. Your server starts by extracting words and bigrams from a given input file to form the initial model, then waits for clients to connect via TCP. Child threads are assigned to incoming TCP connections from these clients, with each child thread responsible for implementing the application-layer protocol between server and client. Through this protocol, clients can add more data to the model or request phrases be generated from the model.

Bigram-based predictive model

A *bigram* is any pair of adjacent words in a given input. For our purposes, all characters between adjacent words is simply ignored. When your server process starts up, it first reads the input file specified by the first command-line argument. Somewhat similar to Homework 1, your task is to extract each valid word from the input file and store it in a dynamically allocated structure that serves as our model. Consider using a `struct` here.

Note that a valid word is defined as any consecutive set of one or more alpha characters (as confirmed via the `isalpha()` library function) that may also contain at most one apostrophe character that is not the first or last character. Convert all characters to lowercase. **(v1.1)** Assume that the maximum word length is 32 bytes.

For each valid word, you must also keep track of the subsequent word, i.e., each bigram. Also keep track of the number of occurrences of each bigram. As an example, consider the following input file, which contains 17 words and therefore 16 bigrams:

```
The quick brown fox jumps over the lazy dog--WOW, I didn't know a fox could jump.
```

Bigrams in this example are: `"The quick"`; `"quick brown"`; `"brown fox"`; `"fox jumps"`; etc.

After processing this input, your memory structure should be organized to map each word to its adjacent subsequent word. Below is how this memory structure would look, with the `1` indicating the number of bigram occurrences encountered thus far.

```
the    --> lazy  1       jumps  --> over 1        didn't --> know 1
       --> quick 1       over   --> the 1         know   --> a 1
quick  --> brown 1       lazy   --> dog 1         a      --> fox 1
brown  --> fox 1         dog    --> wow 1         could  --> jump 1
fox    --> could 1       wow    --> i 1           jump
       --> jumps 1       i      --> didn't 1
```

Continuing the above example, consider the following additional input received later from a client:

```
The quick dog jumps over the moon.
```

Processing this sentence as a new addition to the model, the model expands to the following, with some bigram occurrences incremented to 2 (**(v1.1)** corrected):

```
the    --> quick 2       jumps  --> over 2        know   --> a 1
       --> lazy 1        over   --> the 2         a      --> fox 1
       --> moon 1        lazy   --> dog 1         could  --> jump 1
quick  --> brown 1       dog    --> jumps 1       jump
       --> dog 1                --> wow 1         moon
brown  --> fox 1         wow    --> i 1
fox    --> could 1       i      --> didn't 1
       --> jumps 1       didn't --> know 1
```

Using this model, we can generate phrases from a given start word. Such phrases have a *depth*, which is defined as the maximum number of words in the phrase. If we reach a "dead end" in which no bigram exists (e.g., `"jump"` and `"moon"`), we are unable to go any further. Starting from `"a"`, two such phrases using a depth of 6 are:

```
a fox jumps over the moon
a fox could jump
```

We can also generate multiple phrases based on *breadth*, which is defined as the maximum number of words to choose at each step of phrase generation. Here, we use the ordering shown above based on the number of bigram occurrences; further, we alphabetize words with the same number of bigram occurrences. As an example, phrases starting from `"the"` using a depth of 4 and a breadth of 3 are (**(v1.1)** corrected):

```
the quick brown fox
the quick dog jumps
the quick dog wow
the lazy dog jumps
the lazy dog wow
the moon
```

And if we limit breadth to 2 in this same example and break the "tie" by using `"lazy"` and not getting to `"moon"`, we have (**(v1.1)** corrected):

```
the quick brown fox
the quick dog jumps
the quick dog wow
the lazy dog jumps
the lazy dog wow
```

If we limit breadth to 1, we have:

```
the quick brown fox
```

You should come up with your own data structure to implement the above, but do not focus too much attention and time on fully optimizing your approach.

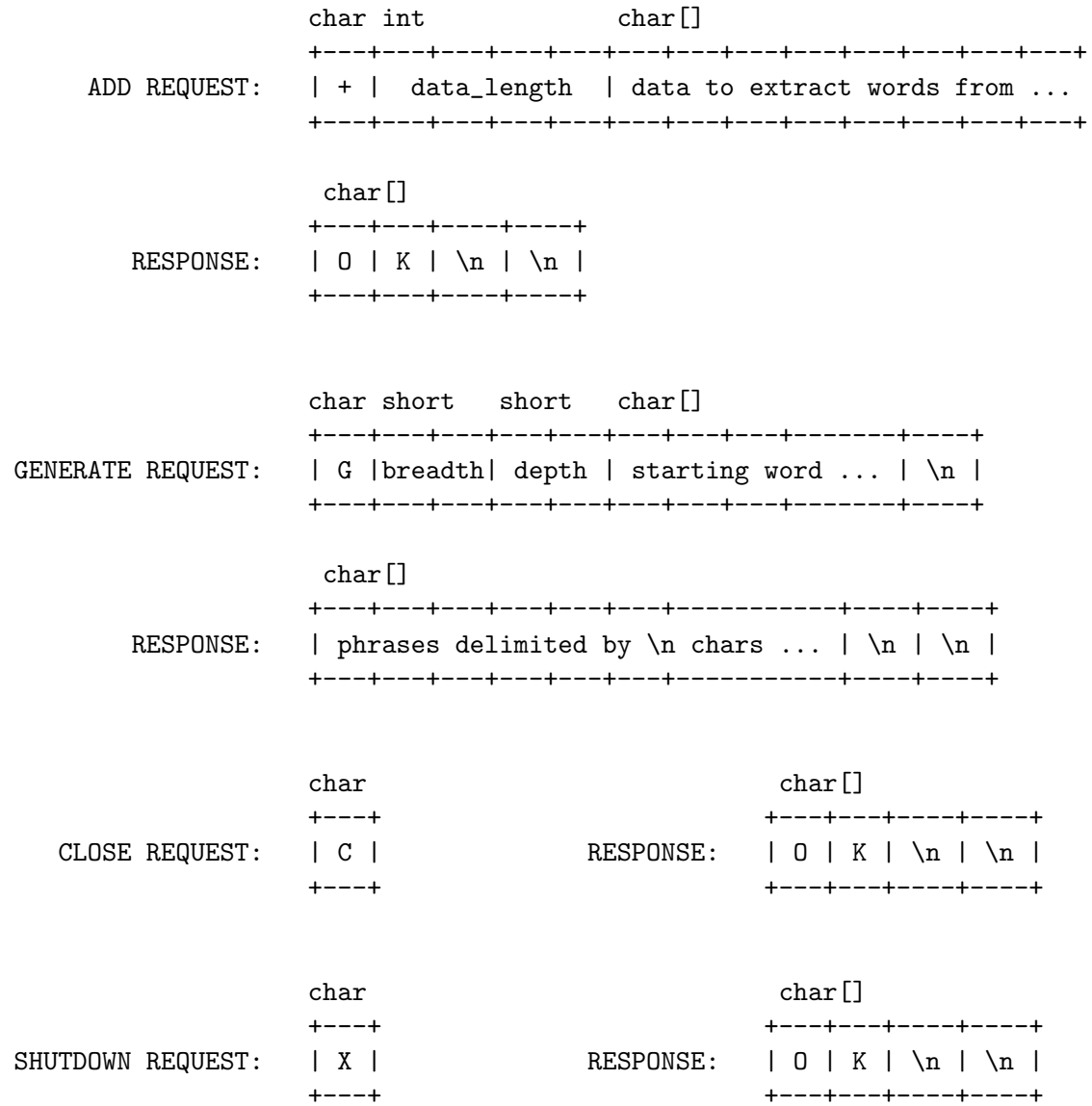
Application-layer protocol

The specifications below describe the application-layer protocol that your TCP server must implement to successfully communicate with multiple clients simultaneously.

As noted earlier, connected clients either add more data to the model or request phrases be generated from the model. A client can also notify the server that it is closing its connection. And a client can also request that the server shutdown, which would certainly not be a good idea in a real-world server.

The first character of the protocol specifies which operation is requested by the client. You could call `recv()` to read this one byte, then determine how many subsequent `recv()` calls you will need.

The protocol is summarized below. All response messages end in two newline characters, i.e., `"\n\n"`. Note that `data_length` is an `int` value in network byte order, and both `breadth` and `depth` are `short` values in network byte order.



Aside from the `int` and `short` values above, you can use `netcat` directly to test your server. To fully test your server, `netcat` can be used with input sent via test files, for example:

```
bash$ netcat localhost 8123 < test01.txt
```

You can also use the `tcp-client.c` example to create a suite of test cases to use to test your server. **(v1.2)** Also check out the `fd-write.c` and `write-data.c` examples. **(v1.3)** And make use of the `hw4-client.c` code.

Command-line arguments

There are two required command-line arguments. As noted earlier, the first command-line argument specifies the path of the input file to extract words and bigrams from. The second command-line argument specifies the port number for your server to use in the `bind()` call. Of course, be sure to also call `socket()` and `listen()` accordingly.

Program execution and required output

To illustrate via an example, you could execute your program as shown below, which includes each of the four types of requests. The server process starts by extracting words from a given `input.txt` file, then runs a TCP server on port `8123`.

```
bash$ cat input.txt
The quick brown fox jumps over the lazy dog--WOW, I didn't know a fox could jump.
bash$ ./hw4.out input.txt 8123
MAIN: extracted 17 words (16 bigrams) from input.txt
MAIN: blocked on accept()
MAIN: new connection established
MAIN: blocked on accept()
THREAD: rcvd '+' ADD request of length 64 bytes
THREAD: extracted 5 bigrams   <========== (v1.3) updated/simplified ==========
THREAD: sent OK response
THREAD: rcvd 'G' GENERATE request with depth 4 and breadth 2
THREAD: sent response with 3 phrases
THREAD: rcvd 'C' CLOSE request
THREAD: sent OK response
MAIN: new connection established
MAIN: blocked on accept()     <========== (v1.1) very likely to appear =======
THREAD: rcvd 'X' SHUTDOWN request
THREAD: sent OK response
MAIN: shutting down after confirming 0 child threads running
bash$
```

Synchronization

Synchronization between all running threads is crucial, which means you must carefully identify the critical sections of your server code. Overall, your main thread blocks on `accept()`, with accepted connections handed off to child threads via `pthread_create()`. Use `pthread_detach()` for all child threads, plus a mutex to synchronize access to a global variable that you use to keep track of the number of active threads.

First, when a child thread is adding more words and bigrams to the model, such writes need to be protected using a mutex. This will ensure that multiple writer threads will not interfere with one another. And this will ensure that any reader threads that are generating phrases do not use incomplete or corrupted data.

Second, your main thread is blocked on the `accept()` call. **(v1.1)** When a child thread receives a shutdown request, the thread should call `shutdown()` on the listener socket descriptor, using a second argument of `SHUT_RDWR`, which will unblock the `accept()` call (but see below). As noted earlier, keep track of the number of child threads running—which is equivalent to the number of active TCP connections—and make sure this value is zero before proceeding with your server shutdown.

(v1.1) More specifically, your main thread will return from the `accept()` call with an error, but the global `errno` will be `EINVAL` in this case. When you detect this in your main thread, use a busy wait to wait until all child threads have completed their tasks. Note that you will need to include `errno.h` (see the `man` page for `errno`).

For all of the above synchronization, you can use a set of binary semaphores; therefore, you must use the `pthread_mutex_lock()` and `pthread_mutex_unlock()` functions.

No square brackets allowed!

As per usual, to emphasize the use of pointers and pointer arithmetic, **you are not allowed to use square brackets** anywhere in your code! If a `'['` or `']'` character is detected, including within comments, Submitty will completely remove that line of code before running `gcc`.

Dynamic memory allocation

Use `malloc()` or `calloc()` to dynamically allocate any memory structures that you need. You might also use `realloc()` to extend the size of these memory structures. Of course, be sure you also use `free()` and have no memory leaks when your server process terminates. Also, be sure no invalid memory references are made, and do not read from or write to any uninitialized memory.

Error handling

In general, if an error is encountered, display a meaningful error message on `stderr` by using either `perror()` or `fprintf()`, then abort further program execution. Only use `perror()` if the given library or system call sets the global `errno` variable.

Error messages must be one line only and use the following format:

```
ERROR: <error-text-here>
```

On Submitty, we might look for the existence of error messages but not specific output.

Submission instructions

To submit your assignment (and perform final testing of your code), we will use Submitty.

To help make sure that your program executes properly, use the techniques below.

First, make use of the `DEBUG_MODE` preprocessor technique illustrated below that helps avoid accidentally displaying extraneous debugging output in Submitty. Here is an example:

```
#ifdef DEBUG_MODE
    printf( "the value of q is %d\n", q );
    printf( "here12\n" );
    printf( "why is my program crashing here?!\n" );
    printf( "aaaaaaaaaaaaaggggggggghhhh no square brackets!\n" );
#endif
```

And to compile this code in "debug" mode, use the `-D` flag as follows:

```
bash$ gcc -Wall -Werror -g -D DEBUG_MODE hw4.c
```

Second, output to standard output (`stdout`) is buffered. To disable buffered output and see program output if your program crashes, call `setvbuf()` immediately in `hw4()` as follows:

```
setvbuf( stdout, NULL, _IONBF, 0 );
```

You would not generally do this in practice since this can slow down your program, but to ensure you see as much output as possible in Submitty, this can be a good technique to use.